Mock Interview Questions——Xiuqi HU

1. 简述最小树生成算法

https://www.bilibili.com/video/BV1Eb41177d1/?spm_id_from=333.337.search-card.all.click&vd_source=13c9ad0fa3cce6eb2516e0bd6cbbe9f5

中文回答：

最小生成树（Minimum Spanning Tree, MST）算法用于在一个带权无向连通图中找到一棵包含所有顶点、边权总和最小的生成树。常用的算法有 Prim 算法和 Kruskal 算法。Prim 算法从一个起始顶点出发，每次选择与当前树相连且权重最小的边，逐步扩展生成树；Kruskal 算法则将所有边按权值从小到大排序，依次选择边，若该边连接的两个顶点不在同一连通分量中（即不构成环），则将其加入生成树，直到包含所有顶点。

English Translation:

Question 1: Briefly describe the minimum spanning tree algorithm.

Answer:

The Minimum Spanning Tree (MST) algorithm finds a tree that connects all vertices in a weighted, undirected, connected graph with the minimum total edge weight. Two common algorithms are Prim's and Kruskal's. Prim's algorithm starts from an arbitrary vertex and repeatedly adds the lowest-weight edge that connects the current tree to a new vertex. Kruskal's algorithm sorts all edges by weight in ascending order and iteratively adds the next smallest edge, provided it does not form a cycle (i.e., the two vertices it connects are in different connected components), until all vertices are included.

---

2. 简述 DFS 和 BFS 的实现逻辑和差别

中文回答：

DFS（深度优先搜索）采用"一条路走到黑"的策略，通常用递归或显式栈实现：从起点出发，沿着一条路径尽可能深入，直到无法继续为止，然后回溯尝试其他路径。BFS（广度优先搜索）则采用"层层扩展"的策略，使用队列实现：从起点开始，先访问所有直接相邻的节点，再依次访问这些节点的邻居，按层推进。两者的主要区别在于：DFS 空间复杂度通常较低（取决于最大深度），但不一定找到最短路径；BFS 保证在无权图中找到从起点到目标的最短路径，但空间开销较大（需存储整层节点）。

English Translation:

Question 2: Briefly describe the implementation logic and differences between DFS and BFS.

Answer:

Depth-First Search (DFS) follows a "go as deep as possible" strategy and is typically implemented using recursion or an explicit stack: it starts from a source node and explores as far as possible along each branch before backtracking. Breadth-First Search (BFS) uses a "layer-by-layer expansion" approach and is implemented with a queue: it visits all immediate neighbors of the start node first, then their neighbors, progressing level by level. The key differences are: DFS generally uses less memory (proportional to the maximum depth) but does not guarantee the shortest path; BFS guarantees the shortest path in an unweighted graph but requires more memory to store all nodes at the current level.

---

3. 简要描述 Python 中几种数据结构的特性

中文回答：

Python 中常用的数据结构及其特征如下：

1. 列表（list）：有序、可变、允许重复元素。支持通过索引访问、插入、删除等操作，底层基于动态数组实现，适合频繁访问但插入/删除（尤其在头部）效率较低。
2. 元组（tuple）：有序、不可变、允许重复元素。一旦创建无法修改，访问速度快，常用于表示固定结构的数据（如坐标、RGB 值等）。
3. 字典（dict）：无序（Python 3.7+ 保持插入顺序）、可变、键唯一。基于哈希表实现，支持 O(1) 平均时间复杂度的查找、插入和删除，适用于键值映射场景。
4. 集合（set）：无序、可变、元素唯一。同样基于哈希表，支持高效的成员检测、去重和集合运算（如并集、交集），但不支持索引访问。

这些数据结构各有适用场景，合理选择能显著提升程序效率与可读性。

English Translation:

Question: Briefly describe the characteristics of several data structures in Python.

Answer:

Common Python data structures and their key characteristics are as follows:

1. List: Ordered, mutable, and allows duplicate elements. Supports indexing, insertion, and deletion. Implemented as a dynamic array, it enables fast random access but relatively slower insertions/deletions at the beginning.
2. Tuple: Ordered, immutable, and allows duplicates. Once created, it cannot be modified. Tuples offer fast access and are often used for fixed data structures (e.g., coordinates, RGB values).
3. Dictionary (dict): Insertion-ordered (as of Python 3.7), mutable, and keys are unique. Implemented using a hash table, it provides average O(1) time complexity for lookups, insertions, and deletions—ideal for key-value mapping.
4. Set: Unordered, mutable, and contains only unique elements. Also hash-table-based, it supports efficient membership testing, deduplication, and set operations (e.g., union, intersection), but does not support indexing.

Choosing the appropriate data structure based on use case significantly improves both performance and code clarity.

Mock Interview Questions——TANG Yirui

1. 请在白板上实现快速排序（Quicksort）算法。

中文回答：

Base case（退出条件）：如果数组长度小于 2，直接返回，因为已经排好序了

选择 pivot：这里选第一个元素作为基准

分区（Partition）：把小于等于 pivot 的放 left，大于 pivot 的放 right

递归合并：递归排序 left 和 right，然后按 left + pivot + right 的顺序合并

时间复杂度：

平均情况：O(n log n) - 如果 pivot 选得好

最坏情况：O(n²) - 比如数组已经排序，像[5,4,3,2,1,0]这样
English Translation:
Question 1: Please implement the Quicksort algorithm on the whiteboard.
Answer:

```
def quicksort(array):
    if len(array) < 2:
        return array
    pivot = array[0]
    left = [i for i in array[1:] if i <= pivot]
    right = [i for i in array[1:] if i > pivot]

    return quicksort(left) + [pivot] + quicksort(right)
```

The algorithm works as follows:
Base case: If the array has less than 2 elements, it's already sorted, so return it
Choose pivot: Here we choose the first element as the pivot
Partition: Put elements ≤ pivot in left, and elements > pivot in right
Recursive merge: Recursively sort left and right, then combine as left + pivot + right
Time Complexity:
Average case: O(n log n) - when pivot is chosen well
Worst case: O(n²) - for example, when the array is already sorted like [5,4,3,2,1,0]
This implementation is very concise - only 6 lines of code! Although quicksort can be O(n²) in the worst case, it's very fast on average, which is why it's widely used in practice.

---

2. 栈和队列的区别 (Stack vs Queue)
中文回答：
栈是 LIFO（后进先出），队列是 FIFO（先进先出）。栈用于函数调用、表达式求值；队列用于 BFS、任务调度。
English Translation:
Stack is LIFO (Last In First Out), Queue is FIFO (First In First Out). Stacks are used for function calls and expression evaluation; queues are used for BFS and task scheduling.

---

3. 什么是递归？(What is Recursion?)
中文回答：
递归就是函数调用自己。基本思想是：
1. 把大问题分解成更小的同类问题
2. 有一个退出条件（base case）
3. 递归调用处理小问题

递归在树的遍历、排序算法（归并排序、快速排序）中都很常用。
English Translation:
Recursion is when a function calls itself. The basic idea is:
1. Break a big problem into smaller similar problems

2. Have an exit condition (base case)
3. Recursively call to handle smaller problems

Recursion is commonly used in tree traversal and sorting algorithms like merge sort and quicksort.

---

4. 选择使用链表而不是数组?

中文回答:

当需要频繁插入和删除操作，而不需要频繁随机访问时，我会选择链表。

原因对比:

- 数组:
  - 插入/删除：O(n)（需要移动元素）
  - 随机访问：O(1)
- 链表:
  - 插入/删除：O(1)（只需改变指针）
  - 访问：O(n)（需要遍历）

English Translation:

Question: When would you choose to use a linked list over an array?

Answer:

I would choose a linked list when the application requires frequent insertions and deletions but doesn't need frequent random access.

Comparison:

- Array:
  - Insert/Delete: O(n) - need to shift elements
  - Random Access: O(1)
- Linked List:
  - Insert/Delete: O(1) - just change pointers
  - Access: O(n) - need to traverse

---

5. 什么是 Big O notation? 请举例说明时间复杂度为 O(n log n)和 O(n²)的算法

中文回答:

Big O notation 是描述算法时间复杂度上界的数学符号，表示算法在最坏情况下的运行时间。

实际例子:

- O(n²)算法：冒泡排序、插入排序
- O(n log n)算法：归并排序、快速排序（平均）、堆排序

English Translation:

Question: What is Big O notation? Please provide some algorithms time complexity is O(n log n) and O(n²).

Answer:

Big O notation is a mathematical notation that describes the upper bound of an algorithm's time complexity, representing its worst-case running time.

Practical Examples:

- O(n²) algorithms: Bubble Sort, Insertion Sort
- O(n log n) algorithms: Merge Sort, Quick Sort (average), Heap Sort

---

6. 请解释二叉树的三种 DFS 遍历方式（前序、中序、后序）的区别，并说明它们的应用场景。
中文回答：
三种 DFS 遍历方式的区别在于访问节点的顺序：
1. 前序遍历（Pre-order）：根-左-右
   应用：复制树、打印树的结构
2. 中序遍历（In-order）：左-根-右
   应用：在二叉搜索树中，中序遍历会得到排序后的结果
3. 后序遍历（Post-order）：左-右-根
   应用：删除树、计算表达式树

English Translation:
Question: Can you explain the differences between the three DFS traversal methods (pre-order, in-order, post-order) for binary trees and their use cases?
Answer:
The three DFS traversal methods differ in the order of visiting nodes:
1. Pre-order: Root-Left-Right
   Use cases: Copying trees, printing tree structure
2. In-order: Left-Root-Right
   Use cases: For Binary Search Trees, in-order traversal gives sorted results
3. Post-order: Left-Right-Root
   Use cases: Deleting trees, evaluating expression trees

---

7. 什么是动态规划?
中文回答：动态规划是一种通过将问题分解为子问题并存储子问题结果来优化的方法。
English Translation:
Question: What is Dynamic Programming?
Answer:
Dynamic Programming is an optimization method that breaks problems into subproblems and stores their results to avoid redundant calculations.

Mock Interview Questions——ZENG Luyang

问题 (Q1)：
请简述单向链表和动态数组相比，各有什么优势和劣势。并举一个适合使用链表而非动态数组的场景。

特性	动态数组 (Dynamic Array)	单向链表 (Singly Linked List)
优势	**随机访问 (Random Access)** 速度快 (O(1))；空间局部性好。	**插入/删除 (Insertion/Deletion)** 操作效率高 (O(1))，无需移动大量元素。
劣势	**插入/删除** 操作效率低 (O(N))，尤其是头部；可能需要重新分配内存。	**随机访问** 速度慢 (O(N))，必须从头遍历。

适合使用链表的场景：
频繁在列表的头部或中间进行插入和删除操作的场景。
示例： 实现一个队列 (Queue) 或栈 (Stack)或者操作系统中的内存管理。
Q1: Advantages and disadvantages of singly linked lists and dynamic arrays. Give one scenario where a linked list is preferable.
Scenario suitable for linked lists:
Frequent insertions or deletions at the head or middle of a list.
Example:
Implementing a queue or stack, or use in OS memory management structures.

问题 (Q2)：

请简述广度优先搜索 (BFS) 算法的基本原理。
原理： BFS 算法从起始节点开始，逐层向外探索，先访问距离起始节点较近的节点，再访问较远的节点。它保证在搜索时，总是先完成对当前层所有邻居节点的访问，然后再进入下一层。

Q2: Basic principle of the Breadth-First Search (BFS) algorithm
BFS starts from the initial node and explores outward level by level.
It visits all neighbors of the current layer before moving on to the next layer.
This guarantees that nodes are visited in order of increasing distance from the start node.

问题 (Q3)：

请对以下数组使用归并排序 (Merge Sort) 进行升序排序。请画出整个过程: [38, 27, 43, 3, 9, 82, 10]
Use Merge Sort to sort the array [38, 27, 43, 3, 9, 82, 10] in ascending order and show the process

1. **最终分解成的所有子数组（长度为 1）：**

$$[38], [27], [43], [3], [9], [82], [10]$$

2. **合并阶段，第一轮结束后（相邻的子数组两两合并）：**
   - $[38], [27] \rightarrow [27, 38]$
   - $[43], [3] \rightarrow [3, 43]$
   - $[9], [82] \rightarrow [9, 82]$
   - $[10]$ (它单独留作一个子数组)

   第一轮合并后的数组状态：

$$[27, 38, 3, 43, 9, 82, 10]$$

问题 (Q4)：

给定一个输入数组：[10, 20, 30]。请分别使用栈 (Stack) 和队列 (Queue) 模拟数据流的入队/入栈和出队/出栈操作，并写出最终输出的数据序列。
使用栈 (Stack)
原则： 后进先出 (LIFO – Last In, First Out)。最终输出序列： 30, 20, 10
使用队列 (Queue)
原则： 先进先出 (FIFO – First In, First Out)。最终输出序列： 10, 20, 30
Q4: Given input array [10, 20, 30], simulate Stack and Queue operations,write down the final output array.

问题 (Q5)：
贪心算法 (Greedy Algorithm) 和 动态规划 (Dynamic Programming) 在解决问题时最主要的区别是什么?
核心差异： 贪心算法在每一步做出局部最优选择，期望达到全局最优，但并不总是有效；动态规划则是通过系统性地考虑和存储所有子问题的解，以确保得到全局最优解。
Q5: Main difference between Greedy Algorithm and Dynamic Programming
Greedy Algorithm:
Makes the best local (immediate) choice at each step, hoping to achieve a global optimum. It does not guarantee optimal results in all problems.
Dynamic Programming:
Systematically solves all subproblems and stores their results to ensure obtaining the global optimal solution.

问题 (Q6)：
二叉搜索树 (BST) 在理想情况下（平衡时）查找、插入、删除的时间复杂度为 O(log n)。但在

最坏情况下，它会退化成什么结构？此时它的时间复杂度会变成多少？为了解决 二叉搜索树 (BST) 的退化问题，引入了哪些平衡二叉搜索树？
BST 退化：在最坏情况下，二叉搜索树会退化成单向链表 (Singly Linked List)。此时，查找、插入、删除操作的时间复杂度从 O(log n)退化为最坏情况下的 O(n)。
AVL 树 和 红黑树
Q6: Worst-case degeneration of a Binary Search Tree (BST) and balanced BST types
Worst case:A BST may degenerate into a singly linked list.
Time complexity becomes: O(n)
To prevent degeneration, balanced BSTs are introduced:
AVL Tree and Red-Black Tree

问题 (Q7):
请简述 P 类问题和 NP 类问题的区别
P 类问题是“容易解决”的问题。
NP 类问题是“容易验证”的问题。
P 类问题是 NP 类问题的子集
Q7: Difference between P and NP problems
P problems:
Problems that can be solved efficiently (in polynomial time).
NP problems:
Problems whose solutions can be verified efficiently (in polynomial time).
Relationship:
P is a subset of NP.

Mock Interview Questions-TU Xinyi

Q1. What is the worst-case time complexity of Quicksort? How does it happen, and how can we avoid it in practice?

Q1. 快速排序（Quicksort）在最坏情况下的时间复杂度是多少？这种情况是如何产生的？在工程实践中我们如何避免它？

- **Answer:**
    - **Complexity:** $O(n^2)$.
    - **Cause:** It happens when the partition is extremely unbalanced, such as when the array is already sorted and the first element is always chosen as the pivot. One recursion branch has size 0, the other $n-1$.
    - **Solution:**
        i. **Randomized Pivot:** Swap the first element with a random element before partitioning.
        ii. **Median-of-Three:** Choose the median of the first, middle, and last elements as the pivot.
- **详细解答：**
    - **最坏情况复杂度：** $O(n^2)$。
    - **产生原因：** 快速排序的性能取决于**划分（Partition）**的平衡性。如果每次选取的基准值（Pivot）都是当前子数组中的最大值或最小值（例如：数组已经是升序排列 `[1, 2, 3, 4, 5]`，而我们总是选择第一个元素作为 Pivot），划分就会极度不平衡。一个子问题的大小为 0，另一个为 $n-1$。递归树退化成一个倾斜的链表，深度为 n。
    - **避免方法：**
        i. **随机化 Pivot (Randomized Quicksort)：** 不固定选第一个元素，而是随机选取一个元素与第一个元素交换后再划分。这使得最坏情况发生的概率极低。
        ii. **三数取中法 (Median-of-Three)：** 取子数组的第一个、最后一个和中间位置的元素，选择这三个数的中位数作为 Pivot。这通常能保证较好的划分平衡性。
    - **平均复杂度：** $O(n\log n)$。

Q2.What is the main difference between an array and a linked list?

Q2. 数组（Array）和链表（Linked List）的主要区别是什么？

Answer:

- **o Memory: An array is a continuous block of memory; Linked lists are fragmented nodes that are connected by pointers.**
- **oAccess: The array supports random access O(1) (via subscript); Linked lists can only access O(n) sequentially.**
- **o Insert/delete: Inserting/deleting an array in the middle requires moving subsequent elements O(n); The linked list simply modifies the pointer O(1) (provided that position has been found).**
- **Size: Arrays are usually fixed in size (dynamic array scaling comes at a cost); Linked lists are dynamic**

答案：

- **内存：** 数组是连续内存块；链表是零散节点，通过指针连接。
- **访问：** 数组支持随机访问 $O(1)$（通过下标）；链表只能顺序访问 $O(n)$。
- **插入/删除：** 数组在中间插入/删除需要移动后续元素 $O(n)$；链表只需修改指针 $O(1)$（前提是已经找到了那个位置）。
- **大小：** 数组通常固定大小（动态数组扩容有代价）；链表是动态的。

Q3. How can you find the k-th smallest element in an unsorted array in o(n) average time?

Q3. 如何在 o(n) 的平均时间复杂度内找到无序数组中第 k 小的元素？（Quickselect 算法）

- **Answer:**
    a. **Algorithm:** Use the `partition` logic from Quicksort.
    b. Pick a pivot and partition the array.
    c. Let p be the pivot's final index.
    d. If $p == k$, return the value.
    e. If $p > k$, recurse only on the **left** side.
    f. If $p < k$, recurse only on the **right** side.
    g. Complexity: Unlike Quicksort, we only process one side. The work is $n + n/2 + n/4 \ldots = O(n)$. Worst case is still $O(n^2)$ if pivots are bad.
- **详细解答：**
    - **算法思路：** 使用与快速排序相同的 `partition` 逻辑。
    - 随机选取 Pivot，将数组分为两部分：左边比 Pivot 小，右边比 Pivot 大。
    - 设 Pivot 的最终下标为 p。
    - 如果 $p == k$，则 Pivot 就是第 k 小元素，直接返回。
    - 如果 $p > k$，说明目标在左边，递归处理左子数组（**不需要**处理右边）。
    - 如果 $p < k$，说明目标在右边，递归处理右子数组。
    - **复杂度：**
        - 与快排不同，Quickselect 每次只处理一半的数据。
        - 平均时间：$n + n/2 + n/4 + \ldots = 2n = O(n)$。

最坏时间：$O(n^2)$（同快排最坏情况）。

Q4. How do you delete a node in a Binary Search Tree (BST), especially if it has two children?

Q4. 在二叉搜索树（BST）中删除一个节点有哪些情况？特别是当该节点有两个子节点时，该如何操作？

- **Answer:**
  - If it's a leaf: Remove directly.
  - One child: Replace node with child.
  - **Two children:** Find the **successor** (minimum value in the right subtree) or **predecessor** (maximum value in the left subtree). Copy the successor's value to the node to be deleted, then recursively delete the successor node (which will have at most one child).
- **详细解答：**
  - **情况 1：叶子节点。** 直接删除，修改父节点指针为 None。
  - **情况 2：只有一个子节点。** 用该子节点替代待删除节点的位置（父节点指针指向子节点）。
  - **情况 3：有两个子节点。** 这是难点。
    i. 在待删除节点的**右子树**中找到**最小值**节点（称为后继 Successor），或者在**左子树**中找到**最大值**节点（称为前驱 Predecessor）。
    ii. 将后继节点的值复制到当前待删除节点。
    iii. 递归地在右子树中删除那个后继节点（后继节点必然没有左子孩子，所以退化为情况 1 或 2）。

Q5. What are the differences between Naive Recursion, Memorized search, and Iteration for computing Fibonacci numbers?

Q5. 斐波那契数列的递归解法、记忆化搜索（Memoization）和迭代解法（DP）有什么区别？

Answer:

- **Naive:** $O(2^n)$ time. Recomputes same subproblems many times.
- **Memoization (Top-down):** $O(n)$ time, $O(n)$ space. Caches results to avoid re-calculation but has recursion stack overhead.
- **Iterative (Bottom-up):** $O(n)$ time, can be $O(1)$ space (only store last two numbers). Avoids recursion overhead.

详细解答：

a. **朴素递归 (Naive Recursion):** `fib(n) = fib(n-1) + fib(n-2)`。存在大量重复计算（例如 `fib(n-2)` 被计算了两次，以此类推）。复杂度为指数级 $O(2^n)$。

b. **记忆化搜索 (Top-down DP):** 仍然使用递归，但维护一个哈希表或数组 `memo`。计算前先查表，有则直接返回，无则计算并存入表。复杂度降为 $O(n)$，但有递归栈的开销。

c. **迭代解法 (Bottom-up DP):** 从 `fib(0)` 和 `fib(1)` 开始，循环计算到 `fib(n)`。只需记录前两个值，空间复杂度可优化至 $O(1)$，时间复杂度 $O(n)$。这是最高效的方法。

Mock interview ——Liao Mengqi

1、 在什么场景下应该选择使用链表而不是数组？**In what scenarios should one opt for linked lists over arrays?**

答案：当你的应用需要频繁地在任意位置进行“插入”和“删除”操作，而较少进行“随机访问”时，链表是更好的选择。

Answer: When your application requires frequent insertion and deletion operations at arbitrary positions, with relatively infrequent random access, a linked list is the superior choice.

2、 二叉搜索树的特点是什么？为什么搜索效率高？**What are the characteristics of BST? Why is its search efficiency high?**

答案：结构性（二叉树，每个节点最多有两个子节点）、排序性（左子树的所有节点的值都小于该节点的值，右子树的所有节点的值都大于该节点的值）、无重复元素。二叉搜索树之所以搜索效率高，是因为它利用其有序的树形结构，在每一步搜索中都实现了“二分”，从而能够快速缩小搜索范围。

Answer: Structural (a binary tree, where each node has at most two child nodes), ordered (all nodes in the left subtree have values less than the node in question, while all nodes in the right subtree have values greater than it), and free of duplicate elements. The high search efficiency of binary search trees stems from their ordered tree structure, which enables a "binary search" at each step, thereby rapidly narrowing the search scope.

3、 什么情况下快速排序会达到最坏的 $O(n^2)$ 时间复杂度？ **Under what circumstances does quick sort achieve its worst-case $O(n^2)$ time complexity?**

答案：最坏情况：1.数组已经有序 2.所有元素都相等 3.每次选择的 pivot 都是最小或最大元素

Answer: Worst-case scenario: 1. The array is already sorted 2. All elements are equal 3. Each selected pivot is the smallest or largest element

4、 红黑树与 AVL 树的主要区别是什么？ **What are the principal differences between red-black trees and AVL trees?**

答案：红黑树和 AVL 树的主要区别在于平衡策略的严格程度不同。AVL 树要求更严格，红黑树要求相对宽松。因此对于查找操作，AVL 树更优，因为树更平衡、高度更低。对于插入删除操作，红黑树更优，需要的旋转操作更少。

Answer：The principal distinction between red-black trees and AVL trees lies in the rigour of their balancing strategies. AVL trees impose stricter requirements, whereas red-black trees are comparatively less stringent. Consequently, AVL trees prove superior for lookup operations, owing to their greater balance and reduced height. For insertion and deletion operations, red-black trees demonstrate greater efficiency, necessitating fewer rotation operations.

5、 在社交网络中，如果你想找到与某人关系最近的朋友，应该用 BFS 还是 DFS？为什么 **On social networks, if you wish to identify the closest friend to a given individual, should you employ BFS or DFS? Why?**

答案：应该使用 BFS。BFS 会从起点开始，一层一层地向外探索。它会先找到所有直接好友，再找好友的好友，以此类推。这保证了当我们第一次找到目标用户时，经历的路径一定是最短的，从而找到的就是关系最近的朋友。而 DFS 会沿着一条路径深入到底，无法保证第一次找到的路径是最短的，因此不适合这个场景。

Answer: BFS should be employed. BFS commences from the starting point, exploring outward layer by layer. It first locates all direct friends, then proceeds to find friends of friends, and so on. This guarantees that when the target user is first identified, the traversed path will invariably be the shortest, thereby locating the closest friend in terms of relationship. Conversely, DFS delves along a single path to its terminus, offering no assurance that the first path found is the shortest. Consequently, it is unsuitable for this scenario.

6、 （算法题）在未排序的数组中找到第 k 个最大的元素 **(Algorithm Problem) Find the kth largest element in an unsorted array.**

```
def find_kth_largest(nums, k):
    # 构建最小堆，保持堆大小为 k
    heap = []
    for num in nums:
        heapq.heappush(heap, num)
        if len(heap) > k:
            heapq.heappop(heap)  # 弹出最小的元素
    return heap[0]  # 堆顶就是第 k 大的元素
```

7、 （算法题）给你二叉树的根节点 root，返回其节点值的层序遍历。 **(Algorithm Problem) Given the root node root of a binary tree, return a level order traversal of its node values.**

```
def levelOrder(root):
    if not root:
        return []

    result = []
```

```
    queue = deque([root])

    while queue:
        level_size = len(queue)
        level = []

        for _ in range(level_size):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)

        result.append(level)

    return result
```

Mock interview questions ——WANG Ziqi

1. Can you compare procedural, functional, and Object-Oriented programming paradigms?
比较一下过程式、函数式和面向对象编程范式

- **Procedural** programming centers on functions and step-by-step execution — focusing on *how* to accomplish a task — and suits simple, linear workflows.
- **Object-oriented** programming (OOP) binds data and behavior into objects via encapsulation, inheritance, and polymorphism — emphasizing *who* performs the action — making it ideal for large, maintainable systems.
- **Functional** programming relies on pure functions and immutable data to avoid side effects, focusing on *what* the result should be — offering strong advantages in concurrency, data transformation, and reliability-critical contexts.

面向过程编程以函数和流程为核心，强调“怎么做”，适合简单、线性的任务；
面向对象编程通过封装、继承和多态，将数据与行为绑定为对象，强调“谁来做”，适合构建大型、可维护的系统；
函数式编程则以纯函数和不可变数据为基础，避免副作用，强调“要什么结果”，在并发处理、数据转换和可靠性要求高的场景中优势显著。

---

2. What are the time complexities for common operations (insert, delete, search) in arrays vs. linked lists? Provide scenarios where one is preferred over the other.
数组和链表操作的时间复杂度

For arrays, index-based access and search (via binary search if sorted) take *O(1)* and *O(log n)* respectively, but insertion/deletion often requires shifting elements, averaging *O(n)*.
In linked lists, insertion/deletion at the head is *O(1)*, but search and arbitrary-position operations require traversal, all *O(n)*.
Thus, we often prefer array for frequent random access (e.g., database indexes); and prefer linked list for frequent inserts/deletes (e.g., task queues).
在数组中，按索引访问和查找（已排序时二分查找）分别为 *O(1)* 和 *O(log n)*，但插入/删除（非尾部）需移动元素，平均 *O(n)*；链表（Linked List） 中，首部插入/删除为 *O(1)*，但查找和任意位置插入/删除需遍历，均为 *O(n)*。

---

3. AVL trees and red-black trees are both self-balancing binary search trees. Please describe their primary strategies for maintaining balance and compare their performance trade-offs in insertion/deletion operations and search operations.
AVL 树和红黑树都是自平衡二叉搜索树。请描述它们维持平衡的主要策略，并比较它们在插入/删除操作和查找操作上的性能权衡.

- AVL Tree maintains strict balance by using a balance factor (height of left subtree - height of right subtree) for each node. This factor must be in {-1, 0, 1}. After an

insertion or deletion, it checks the balance factors up the tree and performs rotations (single or double) to restore the balance if the factor becomes -2 or 2.

- ➤ Red-Black Tree maintains approximate balance by enforcing five key properties rooted in node colors. Balance is restored through recoloring and rotations.

Performance Trade-off:

- Lookup (Search): AVL Tree is faster. Because it is more strictly balanced, the tree is shallower, leading to fewer comparisons on average.
- Insertion/Deletion: Red-Black Tree is faster. It requires at most two rotations per insertion and three for deletion to fix violations. AVL trees may require more rotations to maintain strict height balance, especially during deletions.

AVL 树 - 平衡策略： 它通过为每个节点维护一个平衡因子（左子树高度 - 右子树高度）来保持严格平衡。该因子必须在{-1, 0, 1}范围内。在插入或删除后，它会向上检查树的平衡因子，如果因子变为-2 或 2，则执行旋转（单旋或双旋）以恢复平衡。

红黑树 - 平衡策略： 它通过强制执行基于节点颜色（红/黑）的五条关键属性来维持近似平衡，平衡通过重新着色和旋转来恢复。

- 性能权衡：

查找： AVL 树更快。 因为它更严格地平衡，树更浅，导致平均比较次数更少。

插入/删除： 红黑树更快。 它每次插入最多需要两次旋转，删除最多需要三次旋转来修复违规，并且其平衡规则更具宽容性。AVL 树可能需要更多的旋转来维持严格的高度平衡，尤其是在删除操作期间。

4: What are the five properties of Red-Black trees? 红黑树的五个属性

(1) Nodes red or black. (2) Root black. (3) Red nodes have black children (no two reds adjacent). (4) Every path from node to leaf has same black nodes ensuring black-height balanced. (5) Leaves (NIL) black.

(1)节点红或黑。(2)根黑。(3)红节点的孩子黑。(4)从节点到叶子的每条路径有相同黑节点(黑色高度平衡)。(5)叶子(NIL)黑。

5: Red-black trees have one crucial rule stipulates that "all simple paths from any node to each of its leaf nodes contain the same number of black nodes". What does this rule guarantee?

红黑树一条关键规则是”从任意节点到其每个叶子节点的所有简单路径都包含相同数量的黑色节点"。这个规则保证了什么？

This rule guarantees that the tree is approximately balanced. It ensures that no path is more than twice as long as any other path.

Because the shortest possible path from the root to a NIL node is one that is entirely composed of

black nodes. The longest path would be one that alternates between red and black nodes (due to the rule "no two consecutive red nodes"). Since both the shortest and longest paths must have the same black height, the longest path cannot exceed twice the length of the shortest path. This bound on the longest path is what ensures the O (log n) time complexity for operations.

中文解释：
这条规则保证了树是近似平衡的。它确保了没有任何一条路径会比其他路径长两倍以上
原因:从根节点到 NIL 叶子节点的最短路径可能是全部由黑色节点组成的路径。最长的路径则是红黑节点交替的路径(因为"不能有两个连续的红色节点"规则)。由于最短路径和最长路径必须具有相同的黑高，所以最长路径在节点数量上不会超过最短路径的两倍。这种对最长路径的限制正是确保操作时间复杂度为 O(log n)的关键。

6：Explain BFS and DFS, then compare them

BFS explores a graph level by level. It starts at the root and visits all its immediate neighbors first. Then, it moves to the neighbors of those neighbors, and so on. It uses a Queue (First-In, First-Out). You process nodes in the exact order you discover them.

DFS explores a graph by going as deep as possible down one branch before backtracking. It starts at the root and follows one path to its end, then backtracks and follows the next path. It uses a Stack (Last-In, First-Out), which is often implemented using recursion.

If you need the shortest path or level-based results, use **BFS**. If you need to explore all possibilities, check for connectivity deeply, or are concerned about memory, use **DFS**.

广度优先搜索（BFS）逐层探索图结构。它从根节点开始，首先遍历所有直接邻居节点，随后转向这些邻居节点的邻居节点，依此类推。该算法采用队列（先进先出）机制，节点处理顺序严格遵循发现顺序。 深度优先搜索（DFS）则通过沿单一分支尽可能深入探索后回溯的方式遍历图结构。它从根节点开始，沿单一路径探索至尽头，再回溯至上层节点探索下一条路径。该算法采用栈结构（后进先出），通常通过递归实现。 若需最短路径或基于层级的结果，请选用广度优先搜索；若需探索所有可能性、深度验证连通性，或需处理内存问题，则应采用深度优先搜索

---

7. Please outline the two conditions for a correct greedy algorithm. 贪心算法正确的两个条件

For a greedy algorithm to be correct and yield the global optimum, it must satisfy two conditions:

First, A locally optimal choice at each step leads to a globally optimal solution. This is the key difference from DP, which considers all choices.

Second condition is same as DP, that is the overall optimal solution contains within it optimal solutions to subproblems.

要使贪心算法正确且能得到全局最优解，必须满足两个条件：

首先，在每一步选择局部最优解时，最终得到的解必须是全局最优解。这与考虑所有选择的 DP 存在根本区别。 第二个条件与 DP 相同，即整体最优解必须包含其子问题的最优解。

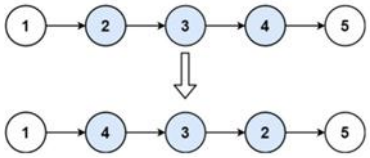
---

2 个代码题：

Q1：Let's work on a linked list problem. You are given the head of a singly-linked list and two integers, left and right, where left <= right. Your task is to reverse the nodes of the list from position left to position right (positions are 1-indexed), and return the modified list.
For example:

input：head = [1,2,3,4,5], left = 2, right = 4

output：[1,4,3,2,5]



给你单链表的头指针 head 和两个整数 left 和 right ，其中 left <= right 。请你反转从位置 left 到位置 right 的链表节点，返回 反转后的链表.

```
class Solution:
   def reverseBetween(self, head: ListNode, left: int, right: int) ->
ListNode:
      dummy_node = ListNode(-1)
      dummy_node.next = head
      pre = dummy_node
      for _ in range(left - 1):
         pre = pre.next

      cur = pre.next
      for _ in range(right - left):
         next = cur.next
         cur.next = next.next
         next.next = pre.next
         pre.next = next
      return dummy_node.next
```

Question 2 (Maximum Depth of Binary Tree)二叉树的最大深度:

Given the root of a binary tree, return its maximum depth.
给定一个二叉树的根节点，返回其最大深度。二叉树的最大深度是从根节点到最远叶子节点的最长路径上的节点数

```python
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def maxDepth(self, root: TreeNode) -> int:
        if not root: return 0
        left_depth = self.maxDepth(root.left)
        right_depth = self.maxDepth(root.right)
        return max(left_depth, right_depth) + 1
```

A binary tree's maximum depth is the number of nodes along the longest path from the root node down to the farthest leaf node.

Mock interview questions ——XuShurui

Question 1: Heap Sort vs Quick Sort with Example / 堆排序 vs 快速排序（含示例）

中文版本

问题：请比较堆排序和快速排序的时间复杂度和空间复杂度，并说明它们在实际应用中的优缺点，并举例说明。

详细回答：

堆排序（Heap Sort）和快速排序（Quick Sort）都是常用的排序算法，但它们在实现方式和性能上有显著差异。

时间复杂度：堆排序在最好、平均和最坏情况下都是 O(n log n)，而快速排序平均是 O(n log n)，但最坏情况下可能退化到 $O(n^2)$，通常发生在每次选取的 pivot 都不理想时。

空间复杂度：堆排序是原地排序，空间复杂度为 O(1)；快速排序由于递归调用，需要 O(log n) 的栈空间。

实际应用：快速排序通常比堆排序快，因为它的常数因子更小，且访问模式具有良好的缓存局部性；堆排序虽然在理论上稳定，但由于访问模式不连续，缓存利用率差，实际速度较慢。

示例：对数组 [9, 4, 7, 1, 3] 使用快速排序，pivot=3，分区后左侧 [1]，右侧 [9,4,7]，递归继续分区，最终得到 [1,3,4,7,9]。

追问：为什么堆排序理论上很好，但实际应用中不如快速排序常用？答案：因为堆排序的内存访问模式不连续，导致缓存性能差，而快速排序的分区操作具有良好的局部性，运行

速度更快。

English Version

Question: Compare heap sort and quick sort in terms of time/space complexity, explain their pros and cons, and provide an example.

Detailed Answer:

Heap Sort and Quick Sort are both widely used sorting algorithms, but they differ significantly in implementation and performance.

Time Complexity: Heap Sort runs in O(n log n) in the best, average, and worst cases. Quick Sort averages O(n log n) but can degrade to $O(n^2)$ in the worst case when pivots are poorly chosen.

Space Complexity: Heap Sort is in-place with O(1) extra space, while Quick Sort requires O(log n) stack space due to recursion.

In practice: Quick Sort is usually faster because of smaller constant factors and better cache locality. Heap Sort, although theoretically stable, suffers from poor cache performance and is slower in real-world scenarios.

Example: For array [9, 4, 7, 1, 3], Quick Sort with pivot=3 partitions into [1] and [9,4,7], recursively sorts them to get [1,3,4,7,9].

Follow-up: Why is Heap Sort less used in practice? Answer: Its memory access pattern is scattered, leading to poor cache utilization, whereas Quick Sort's partitioning exhibits good locality and runs faster.

Question 2: BST Traversal with Example / BST 遍历（含示例）

中文版本

问题：请解释前序、中序、后序遍历的区别，并说明中序遍历在 BST 中的作用，并举例说明。

详细回答：

二叉搜索树（BST）的遍历方式包括前序、中序和后序遍历，它们的访问顺序不同：

前序遍历：根 → 左 → 右；中序遍历：左 → 根 → 右；后序遍历：左 → 右 → 根。

在 BST 中，中序遍历的作用是输出升序序列，因此常用于排序和范围查询。

示例：对于 BST 节点值 {5,3,7,2,4,6,8}，中序遍历输出 [2,3,4,5,6,7,8]。

伪代码（递归中序遍历）：

```
def inorder(node):
    if node is None: return
    inorder(node.left)
    print(node.value)
    inorder(node.right
```

非递归版本（使用栈）：

```
stack = []
curr = root
while curr or stack:
    while curr:
        stack.append(curr)
        curr = curr.left
    curr = stack.pop()
    print(curr.value)
    curr = curr.right
```

English Version

Question: Explain pre-order, in-order, and post-order; describe in-order's role in BST, and provide an example.

Detailed Answer:

Binary Search Tree (BST) supports three main traversal orders: pre-order, in-order, and post-order.

Pre-order: Root → Left → Right; In-order: Left → Root → Right; Post-order: Left → Right → Root.

In BST, in-order traversal outputs elements in ascending order, making it useful for sorting and

range queries.

Example: For BST with nodes {5,3,7,2,4,6,8}, in-order traversal outputs [2,3,4,5,6,7,8].

Recursive pseudo-code (In-order):

```
def inorder(node):
    if node is None: return
    inorder(node.left)
    print(node.value)
    inorder(node.right
```

Iterative version using stack:

```
stack = []
curr = root
while curr or stack:
    while curr:
        stack.append(curr)
        curr = curr.left
    curr = stack.pop()
    print(curr.value)
    curr = curr.right
```

Question 3: BFS vs DFS with Example / BFS vs DFS（含示例）

中文版本

问题：请比较 BFS 和 DFS 的特点，并说明它们在图算法中的典型应用，并举例说明。

详细回答：

BFS（广度优先搜索）和 DFS（深度优先搜索）是两种常用的图遍历算法，它们的策略和应用场景不同：

BFS：按层次遍历，使用队列实现，适用于无权图的最短路径、层次遍历。

DFS：沿路径深入，使用栈或递归实现，适用于拓扑排序、检测环、连通分量。

示例：在图 {A:[B,C],B:[D],C:[E],D:[],E:[]} 中，BFS 从 A 开始访问顺序 [A,B,C,D,E]，DFS 访问顺序 [A,B,D,C,E]。

为什么 BFS 可以找到最短路径？因为 BFS 按层扩展，首次到达目标节点时路径最短；DFS 可能先走很长的路径，不能保证最短。

English Version

Question: Compare BFS and DFS, explain typical applications, and provide an example.

Detailed Answer:

Breadth-First Search (BFS) and Depth-First Search (DFS) are two common graph traversal algorithms with different strategies and use cases:

BFS: Explores level by level using a queue; suitable for shortest path in unweighted graphs and level-order traversal.

DFS: Goes deep along a path using a stack or recursion; suitable for topological sorting, cycle detection, and connected components.

Example: For graph {A:[B,C],B:[D],C:[E],D:[],E:[]}, BFS from A visits [A,B,C,D,E], DFS visits [A,B,D,C,E].

Why does BFS find the shortest path? Because BFS expands nodes by levels, so the first time it reaches the target node, the path is the shortest. DFS may go deep along a long path first, so it cannot guarantee the shortest path.

Question 4: Merge Sort / 归并排序（Merge Sort）

中文版本

问题：请解释归并排序的原理、时间复杂度，并举例说明。

详细回答：

归并排序（Merge Sort）是一种分治算法，将数组递归地分成两半，分别排序后再合并。

时间复杂度：O(n log n)，空间复杂度：O(n)，因为需要额外数组存储合并结果。

示例：对数组 [8,4,5,2]，分成 [8,4] 和 [5,2]，分别排序为 [4,8] 和 [2,5]，合并得到 [2,4,5,8]。

伪代码：

```
def merge_sort(arr):
    if len(arr)<=1: return arr
    mid=len(arr)//2
    left=merge_sort(arr[:mid])
    right=merge_sort(arr[mid:])
    return merge(left,right)
```

English Version

Question: Explain the principle of merge sort, its time complexity, and provide an example.

Detailed Answer:

Merge Sort is a divide-and-conquer algorithm that recursively splits the array into halves, sorts each half, and merges them.

Time complexity: O(n log n), space complexity: O(n) due to extra array for merging.

Example: For array [8,4,5,2], split into [8,4] and [5,2], sort them to [4,8] and [2,5], merge to get [2,4,5,8].

Pseudo-code:

```
def merge_sort(arr):
    if len(arr)<=1: return arr
    mid=len(arr)//2
    left=merge_sort(arr[:mid])
    right=merge_sort(arr[mid:])
    return merge(left,right)
```

1. 数组和链表

问题：

数组和链表是最基础的数据结构。在中间位置插入一个元素，分别怎么实现?

回答：

数组：需先将插入位置后的所有元素向后移动一位（时间复杂度 O (n)），再插入新元素。

链表：只需修改插入位置前后节点的指针（时间复杂度 O (1)），无需移动元素。

核心差异：数组移动元素效率低，链表指针操作效率高。

Q: Arrays and linked lists are the most basic data structures. How to insert an element at the middle position respectively?

A: Array: First, shift all elements after the insertion position one position backward (time complexity O(n)), then insert the new element.

Linked list: Only need to modify the pointers of the nodes before and after the insertion position (time complexity O(1)), no need to move elements.

Core difference: Arrays are inefficient due to element shifting, while linked lists are efficient with pointer operations.

---

2. 插入排序 vs 冒泡排序

问题：

插入排序和冒泡排序都是 O(n^2)时间复杂度，实际用的时候，哪种更适合 “近乎有序的数组”？比如数组 [1,3,2,4,5,7,6]，选插入还是冒泡？说明理由。

回答：

选择插入排序。

近乎有序时，插入排序的比较和移动次数极少（接近 O (n)），因为元素只需插入到相邻的正确位置。

冒泡排序仍需频繁交换相邻元素（即使有序也需遍历确认），效率更低。

[1,3,2,4,5,7,6] 中，插入排序仅需移动 2 和 6，而冒泡排序需多次交换。

Q: Both insertion sort and bubble sort have a time complexity of $O(n^2)$. In practice, which one is more suitable for "nearly sorted arrays"? For example, for the array [1,3,2,4,5,7,6], should we choose insertion sort or bubble sort? Please explain the reason.

A: Choose insertion sort.

For nearly sorted arrays, insertion sort requires very few comparisons and shifts (close to O(n)), because elements only need to be inserted into the adjacent correct position.

Bubble sort still needs frequent swaps of adjacent elements (even needs traversal confirmation for sorted arrays), resulting in lower efficiency.

In [1,3,2,4,5,7,6], insertion sort only needs to shift 2 and 6, while bubble sort requires multiple swaps.

---

3. 快速排序的“最好 vs 最坏”

问题：

快速排序的平均时间复杂度是 O(log n)，但什么情况下会变成 O(n^2)？怎么简单优化（比如选基准值的方式 ）避免这种最坏情况？

回答：

最坏情况：

当数组已有序或逆序，且每次选择最左 / 右端为基准值时，划分会极度不均（一边为空，另一边为 n-1 个元素），时间复杂度退化为 O (n^2)。

优化方式：

随机选择基准值（避免固定位置导致的最坏情况）。

三数取中法（选择左端、中间、右端的中位数作为基准）。

优化后：平均时间复杂度仍为 O (n log n)，最坏情况概率极低。

Q: The average time complexity of quicksort is O(n log n), but in what cases will it degenerate to $O(n^2)$? How to simply optimize (such as the way of selecting pivot) to avoid this worst-case scenario?

A: Worst-case scenario:

When the array is already sorted or reverse-sorted, and the leftmost/rightmost element is selected as the pivot each time, the partition will be extremely uneven (one side is empty, and the other side has n-1 elements), leading to a time complexity of $O(n^2)$.

Optimization methods:

Randomly select the pivot (avoid the worst case caused by fixed positions).

Median-of-three method (select the median of the left-end, middle, and right-end elements as the pivot).

After optimization: The average time complexity remains O(n log n), and the probability of the worst case is extremely low.

---

4. 链表的“删除节点”

问题：

单向链表中，删除某个节点（已知该节点）需要注意什么？假设链表是 1→2→3→4，要删除节点 3，步骤是什么？（在平板上绘出流程图，不需要说。）

回答：

单向链表（删除节点 3）：

需找到节点 3 的前驱节点（节点 2）。

令 2.next = 3.next（跳过节点 3）。

注意：需确保节点 3 不是尾节点（否则 3.next 为 null）。

Q: What should be noted when deleting a node (the node is known) in a singly linked list? Assuming the linked list is 1→2→3→4, what are the steps to delete node 3? (Draw a flow chart on the tablet, no need to explain verbally.)

A: Singly linked list (deleting node 3):

First, find the predecessor node of node 3 (node 2).

Set 2.next = 3.next (skip node 3).

Note: Ensure that node 3 is not the tail node (otherwise 3.next is null).

---

5. 链表实现“逆序打印”

问题：

用单向链表存储数据，怎么逆序打印链表内容？要求不能修改链表结构（比如反转链表），说说思路（提示：递归或栈）。（在平板上绘出流程图和简单代码）

回答：

递归：

递归遍历到链表末尾，回溯时打印当前节点值。

例：1→2→3 → 递归到 3 打印 → 回溯打印 2 → 回溯打印 1。

核心：利用递归或栈的“后进先出”特性，避免修改链表结构。

Q: How to print the content of a singly linked list in reverse order without modifying the linked list structure (such as reversing the linked list)? Please explain the idea (hint: recursion or stack). (Draw a flow chart and simple code on the tablet.)

A: Recursion:

Recursively traverse to the end of the linked list, and print the value of the current node during backtracking.

Example: 1→2→3 → recursively reach 3 and print → backtrack and print 2 → backtrack and print 1.

Core: Utilize the "Last-In-First-Out" feature of recursion or stack to avoid modifying the linked list structure.